

claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\agent-ad763e306e25f8f4e.jsonl`  
- SHA-256: `f53582da3ebd5d90fe2e2a8e38fe980d94dff37dfe8d87c237de6db1406ce0fb`  
- Source modified: `2026-06-21T08:33:09+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `subagents`  
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`
```

Transcript

1. user (2026-06-21T08:30:31.548Z)

Review milestone M3 of COMPUTE_DECOUPLED_SERVING in the repo at C:/Users/avidu/Projects/badminton-highlight-indexer. Run `git diff` (working tree vs HEAD) to see the change.

```
M3 = "configurable output/scratch base ? kill hardcoded output/ in the segmenters; DEFAULT stays output/; golden + stub-E2E byte-identical; default-OFF". The change adds  
`backend/utils/paths.py::output_base(config)` (returns `output.output_dir`, default "output")  
and routes 6 hardcoded `os.path.join("output", ...)` scratch/staging/temp sites in  
`backend/pipeline/segmenters/{trajectory_hybrid.py, yolo_hybrid.py, gemini.py}` through it.  
`_chunk_workdir` gained a `base="output"` default param. New tests in  
`tests/test_output_base.py`.
```

Verify ONLY these, grounded in code with file:line:

1. BYTE-IDENTICAL DEFAULT: with the default config (output.output_dir=="output"), does every changed site produce the exact same path as before? Any site where output_base() could return something other than "output" by default (e.g. a code path where config lacks an output section)?
2. CREATE/CLEANUP CONSISTENCY: each scratch dir is created (os.makedirs) and later removed (os.rmdir) ? do BOTH use the same now-config-rooted variable, or could cleanup still point at a literal "output" (leaving an orphan / failing rmdir)?
3. MISSED SITES: are there any OTHER hardcoded production scratch/output literals in these 3 segmenter files (or a shared helper they call) that M3 left behind? (The eval-only `rally\_seg\_eval.py` DEFAULT_MANIFEST is explicitly OUT of scope ? ignore it.)
4. CONFIG ACCESS: `output\_base` uses `config.get("output", {}).get("output\_dir")`. Does this work for BOTH the typed AppConfig (its .get shim returns nested sections as dicts) AND a legacy dict config? Any case it raises instead of falling back to "output"?
5. TEST GAPS: does test_output_base.py actually pin the default-byte-identical AND the custom-output_dir behavior at a real segmenter call site? Anything important untested (e.g. yolo_hybrid or gemini call sites, vs only wasb)?

Report concrete findings with severity (blocker/major/minor/nit) and a fix. A clean bill is a valid result. Do NOT make any edits ? review only.

2. assistant (2026-06-21T08:30:34.009Z)

I'll review milestone M3 of COMPUTE_DECOUPLED_SERVING. Let me start by examining the change.

3. user (2026-06-21T08:30:35.676Z)

```
backend/pipeline/segmenters/gemini.py          | 16 ++++++-----  
backend/pipeline/segmenters/trajectory_hybrid.py |  5 +++--  
backend/pipeline/segmenters/yolo_hybrid.py      |  5 +++--  
backend/utils/paths.py                          | 18 ++++++-----  
4 files changed, 33 insertions(+), 11 deletions(-)
```

4. user (2026-06-21T08:30:36.137Z)

On branch feat/serving-m2-input-backend

Your branch is based on 'origin/feat/serving-m2-input-backend', but the upstream is gone.

(use "git branch --unset-upstream" to fixup)

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: backend/pipeline/segmenters/gemini.py

modified: backend/pipeline/segmenters/trajectory_hybrid.py

modified: backend/pipeline/segmenters/yolo_hybrid.py

modified: backend/utils/paths.py

Untracked files:

(use "git add <file>..." to include in what will be committed)

artifacts/

docs/KHELSUTRA_PRODUCT_OVERVIEW.md

tests/test_output_base.py

no changes added to commit (use "git add" and/or "git commit -a")

---BRANCH---

feat/serving-m2-input-backend

5. assistant (2026-06-21T08:30:38.984Z)

Note: the branch is `feat/serving-m2-input-backend` (not yet pulled/rebased, upstream gone).

I'll review the working-tree diff as requested. Let me look at the full diff and the relevant files.

6. user (2026-06-21T08:30:45.244Z)

diff --git a/backend/pipeline/segmenters/gemini.py b/backend/pipeline/segmenters/gemini.py

index 1992572..696f45c 100644

--- a/backend/pipeline/segmenters/gemini.py

+++ b/backend/pipeline/segmenters/gemini.py

@@ -6,6 +6,7 @@ import concurrent.futures

from typing import List, Dict, Any, Tuple, Callable

from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry

from backend.utils.video import get_video_duration, extract_video_chunk

+from backend.utils.paths import output_base

from backend.sports import sport_registry

from backend.providers import provider_registry

from backend.reporting import run_signals

@@ -13,14 +14,16 @@ from backend.reporting import run_signals

logger = logging.getLogger(__name__)

-def _chunk_workdir(video_id: str) -> str:

- """Per-video chunk staging dir under output/chunks/<video_id>.

+def _chunk_workdir(video_id: str, base: str = "output") -> str:

+ """Per-video chunk staging dir under <base>/chunks/<video_id> (`base` default
+ ``output``

+ keeps the prior literal byte-identical; the caller passes the configured output base so
+ staging follows --output-dir ? COMPUTE_DECOUPLED_SERVING M3).

Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?

otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's

in-flight work. A per-video subdir means each run only ever creates/cleans its own.

"""

- return os.path.join("output", "chunks", video_id)

+ return os.path.join(base, "chunks", video_id)

def _parse_time(val) -> float:

```

@@ -116,8 +119,9 @@ class GeminiPlaySegmenter(BasePlaySegmenter):
    if debug_duration:
        logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration} seconds for
fast processing...")
-        temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
-        os.makedirs("output", exist_ok=True)
+        base = output_base(self.config)
+        temp_trimmed_path = os.path.join(base, f"temp_trimmed_{video_id}.mp4")
+        os.makedirs(base, exist_ok=True)
        if os.path.exists(temp_trimmed_path):
            try:
                os.remove(temp_trimmed_path)
@@ -533,7 +537,7 @@ class GeminiPlaySegmenter(BasePlaySegmenter):
    # Per-video chunk workdir (concurrency isolation): a concurrent run for another
    # video keeps its own dir, so cleanup here never removes another run's chunks.
-    chunks_dir = _chunk_workdir(video_id)
+    chunks_dir = _chunk_workdir(video_id, output_base(self.config))

    if use_chunking:
        ai_segments = self._process_chunked(
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index 1fff3c6..6f4f512 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -27,6 +27,7 @@ import cv2

from backend.pipeline.segmenters.base import BasePlaySegmenter
from backend.utils.video import get_video_duration, extract_video_chunk
+from backend.utils.paths import output_base
from backend.sports import sport_registry
from backend.providers import provider_registry
from backend.pipeline.detectors.base import DetectorRunner
@@ -234,7 +235,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
    # Trajectory detectors process the whole video in one pass (they carry
    # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
    t_start = time.time()
-    out_dir = os.path.join("output", self.family)
+    out_dir = os.path.join(output_base(self.config), self.family)
+    csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
    if not csv_path:
        logger.error("%s produced no trajectory; aborting.", label)
@@ -310,7 +311,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
        logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
                    "(no AI handoff).", len(ai_segments))
    else:
-        handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+        handoff_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")
        os.makedirs(handoff_dir, exist_ok=True)
        logger.info("Phase 3: %s validation on %d candidate windows...",
                    provider_name, len(handoff_indices))
diff --git a/backend/pipeline/segmenters/yolo_hybrid.py
b/backend/pipeline/segmenters/yolo_hybrid.py
index 5c163d8..0d84b6b 100644
--- a/backend/pipeline/segmenters/yolo_hybrid.py
+++ b/backend/pipeline/segmenters/yolo_hybrid.py
@@ -24,6 +24,7 @@ from typing import List, Dict, Any, Tuple
from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
from backend.utils.video import get_video_duration, extract_video_chunk
+from backend.utils.paths import output_base

```

```

from backend.sports import sport_registry
from backend.providers import provider_registry

@@ -265,7 +266,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
    """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
    the provider for semantic boundaries, in parallel. Verbatim move."""
    gemini_segments = []
-    gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+    gemini_dir = os.path.join(output_base(self.config), f"chunks_{provider_name}_handoff")
    os.makedirs(gemini_dir, exist_ok=True)

    logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
@@ -404,7 +405,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
    "min_action_window_duration": idx_cfg.get("min_action_window_duration", 2.0),
    }

-    chunks_dir = os.path.join("output", "chunks_yolo")
+    chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
    os.makedirs(chunks_dir, exist_ok=True)

    # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
diff --git a/backend/utils/paths.py b/backend/utils/paths.py
index 10ac3d6..9b04302 100644
--- a/backend/utils/paths.py
+++ b/backend/utils/paths.py
@@ -1,6 +1,22 @@
    """Path-safety helpers for the API boundary (untrusted request fields)."""
    import os
-    from typing import Optional
+    from typing import Any, Optional
+
+
+def output_base(config: Any) -> str:
+    """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
+    (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``) so the
+    chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` / ``output.output_dir``
+    instead of
+    a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
+    literal.
+
+    This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which may be
+    a
+    ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy dict-shim
+    (``.get("output", {})).get("output_dir")``; any miss or empty value falls back to
+    ``"output"``.
+
+    try:
+        out = config.get("output", {}).get("output_dir")
+    except Exception:
+        out = None
+    return out or "output"

def safe_output_filename(name: str) -> str:

```

7. user (2026-06-21T08:30:45.515Z)

```

1    """Path-safety helpers for the API boundary (untrusted request fields)."""
2    import os
3    from typing import Any, Optional
4
5
6    def output_base(config: Any) -> str:
7        """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
8        (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)

```

```

so the
9         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
10         a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
11
12         This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
13         ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
dict-shim
14         (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``.
15         try:
16             out = config.get("output", {}).get("output_dir")
17         except Exception:
18             out = None
19         return out or "output"
20
21
22     def safe_output_filename(name: str) -> str:
23         """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
24
25         ``os.path.join(output_dir, name)`` silently discards ``output_dir`` when ``name`` is
absolute,
26         and ``..`` segments escape it ? so an unchecked ``output_filename`` is an arbitrary-
write
27         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
28         """
29         if not name or not name.strip():
30             raise ValueError("output_filename must not be empty")
31         if "\x00" in name:
32             raise ValueError("output_filename must not contain a null byte")
33         if "/" in name or "\\" in name:
34             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
35         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
36             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
37         return name
38
39
40     def resolve_under_root(path: str, root: str) -> str:
41         """Return the real path, asserting it is contained under ``root`` (raises ValueError
if not)."""
42         rp = os.path.realpath(path)
43         rr = os.path.realpath(root)
44         try:
45             contained = os.path.commonpath([rp, rr]) == rr
46         except ValueError: # different drives on Windows
47             contained = False
48         if not contained:
49             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
50         return rp
51
52
53     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
54         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
55
56         With no allowed_root (default, single-user local mode) only the null-byte check
applies,
57         so the local 'point it at any of my videos' workflow is preserved. Setting an
58         allowed_root (a future hosted/tenant deployment) enforces containment.
59         """
60         if "\x00" in path:
61             raise ValueError("video_path must not contain a null byte")
62         if allowed_root:

```

```
63         resolve_under_root(path, allowed_root)
64
```

8. user (2026-06-21T08:30:45.944Z)

```
1     """Configurable output/scratch base (COMPUTE_DECOUPLED_SERVING M3).
2
3     The segmenters' chunk-staging / AI-handoff / temp dirs no longer hardcode ``./output`` ?
they
4     root under the configured ``output.output_dir`` (default ``"output"`` , so byte-identical
to the
5     prior literal; a custom ``--output-dir`` / ``output.output_dir`` now actually moves the
scratch).
6     """
7     import os
8     from unittest.mock import MagicMock, patch
9
10    from backend.config.models import AppConfig
11    from backend.pipeline.segmenters.gemini import _chunk_workdir
12    from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
13    from backend.utils.paths import output_base
14
15
16    # ----- output_base() helper
17    def test_output_base_default_is_output():
18        assert output_base(AppConfig()) == "output"          # typed config, built-in
default
19        assert output_base({"output": {"output_dir": "output"}}) == "output"
20
21
22    def test_output_base_honors_custom_output_dir():
23        cfg = AppConfig()
24        cfg.output.output_dir = "/tmp/custom_out"
25        assert output_base(cfg) == "/tmp/custom_out"
26        assert output_base({"output": {"output_dir": "reels"}}) == "reels" # legacy dict-
shim
27
28
29    def test_output_base_falls_back_to_output_on_missing_or_empty():
30        assert output_base({}) == "output"                  # no output section
31        assert output_base({"output": {}}) == "output"      # no output_dir key
32        assert output_base({"output": {"output_dir": ""}}) == "output" # empty -> default
33        assert output_base(None) == "output"                # defensive: bad config
34
35
36    # ----- _chunk_workdir base
37    def test_chunk_workdir_default_base_is_byte_identical():
38        # The no-arg call (used by tests / any legacy caller) is unchanged.
39        assert _chunk_workdir("vidA") == os.path.join("output", "chunks", "vidA")
40
41
42    def test_chunk_workdir_honors_custom_base():
43        assert _chunk_workdir("vidA", "/tmp/out") == os.path.join("/tmp/out", "chunks",
"vidA")
44
45
46    # ----- segmenter call-site
47    def _mock_runner():
48        runner = MagicMock()
49        runner.healthcheck.return_value = (True, "env ok")
50        runner.run_predict.return_value = "out.csv"
51        runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
52        runner.trajectory_to_action_windows.return_value = [{
53            "start": 1.0, "end": 4.0, "active_frame_count": 30,
54            "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1, "chunk_index":
```

```

0,
55     }]
56     return runner
57
58
59     def _run_wasb_capture_out_dir(config):
60         """Run wasb_hybrid through the standard mocked pipeline; return the out_dir passed
to
61         run_predict (Phase 2 trajectory dir)."""
62         runner = _mock_runner()
63         db = MagicMock()
64         db.add_candidate_segment.return_value = 55
65         db.add_play_segment.return_value = 200
66         provider = MagicMock()
67         provider.analyze_video.return_value = (
68             [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}], {})
69         with patch.object(WasbHybridSegmenter, "_build_runner", return_value=(runner,
{"weights_path": "w"})), \
70             patch.object(WasbHybridSegmenter, "_probe_fps_width", return_value=(30.0,
1280.0)), \
71             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
72             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
73             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
74             patch("os.makedirs"), patch("os.environ.get", return_value="key"):
75             preg.get.return_value = provider
76             seg = WasbHybridSegmenter(db, config)
77             seg.process_video("v1", "clip.mp4")
78             return runner.run_predict.call_args[0][1] # 2nd positional arg = out_dir
79
80
81     def test_trajectory_scratch_default_is_literal_output():
82         """Default config: out_dir is `output/wasb` ? byte-identical to the prior
hardcode."""
83         assert _run_wasb_capture_out_dir({"indexing": {}}) == os.path.join("output", "wasb")
84
85
86     def test_trajectory_scratch_follows_custom_output_dir(tmp_path):
87         """A configured output_dir actually moves the trajectory scratch (the latent bug M3
fixes)."""
88         custom = str(tmp_path / "custom_out")
89         out_dir = _run_wasb_capture_out_dir({"indexing": {}, "output": {"output_dir":
custom}})
90         assert out_dir == os.path.join(custom, "wasb")
91

```

9. assistant (2026-06-21T08:30:49.431Z)

Now let me examine the segmenter files in full to look for missed sites and create/cleanup consistency. Let me look at the gemini, trajectory_hybrid, and yolo_hybrid files for all `output` literals and rmdir/makedirs patterns.

10. user (2026-06-21T08:30:51.447Z)

```

9:from backend.utils.paths import output_base
17:def _chunk_workdir(video_id: str, base: str = "output") -> str:
18:    """Per-video chunk staging dir under <base>/chunks/<video_id> (`base` default
`"output"`)
23:    otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
122:        base = output_base(self.config)
124:        os.makedirs(base, exist_ok=True)
127:        os.remove(temp_trimmed_path)
152:        os.remove(temp_trimmed_path)

```

```

186:         os.makedirs(chunks_dir, exist_ok=True)
238:             os.remove(chunk_path)
314:         os.rmdir(chunks_dir)
540:         chunks_dir = _chunk_workdir(video_id, output_base(self.config))

```

11. user (2026-06-21T08:30:53.357Z)

```

30:from backend.utils.paths import output_base
238:     out_dir = os.path.join(output_base(self.config), self.family)
314:         handoff_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")
315:         os.makedirs(handoff_dir, exist_ok=True)
339:             os.remove(w_path)
350:             os.rmdir(handoff_dir)

```

12. user (2026-06-21T08:30:54.553Z)

```

27:from backend.utils.paths import output_base
161:             os.remove(chunk_path)
186:             os.remove(chunk_path)
269:         gemini_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")
270:         os.makedirs(gemini_dir, exist_ok=True)
301:             os.remove(w_path)
314:             os.rmdir(gemini_dir)
408:         chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
409:         os.makedirs(chunks_dir, exist_ok=True)
424:             os.rmdir(chunks_dir)

```

13. assistant (2026-06-21T08:30:58.404Z)

The grep confirms create/cleanup pairs use the same variable in each segmenter. Let me verify the create/cleanup consistency by reading the surrounding context of each makedirs/rmdir pair, and confirm `self.config` is the right attribute (not the typed AppConfig stored elsewhere). Let me read the relevant blocks.

14. user (2026-06-21T08:30:59.192Z)

```

115         debug_duration = self.config.get("indexing", {}).get("debug_duration")
116         video_to_upload = video_path
117         is_trimmed = False
118         temp_trimmed_path = None
119
120         if debug_duration:
121             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
122             base = output_base(self.config)
123             temp_trimmed_path = os.path.join(base, f"temp_trimmed_{video_id}.mp4")
124             os.makedirs(base, exist_ok=True)
125             if os.path.exists(temp_trimmed_path):
126                 try:
127                     os.remove(temp_trimmed_path)
128                 except Exception:
129                     pass
130
131             cmd = [
132                 "ffmpeg", "-y",
133                 "-ss", "0",
134                 "-t", str(debug_duration),
135                 "-i", video_path,
136                 "-c", "copy",
137                 temp_trimmed_path
138             ]
139             logger.debug(f"Running command: {' '.join(cmd)}")

```



```

140         try:
141             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
142             logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
143             video_to_upload = temp_trimmed_path
144             is_trimmed = True
145         except Exception as e:
146             logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
147
148     # Helper to clean up local temp files
149     def cleanup_temp_files():
150         if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
151             try:
152                 os.remove(temp_trimmed_path)
153             except Exception:
154                 pass
155
156     return video_to_upload, cleanup_temp_files
157
158 def _process_chunked(
159     self,
160     video_id: str,
161     video_to_upload: str,
162     sport_profile,
163     provider,
164     video_duration: float,
165     chunk_duration: float,
166     chunk_overlap: float,
167     ai_scale_width: int,
168     chunks_dir: str,
169     cleanup_temp_files: Callable[[], None],
170 ) -> List[Dict[str, Any]]:
171     """--- CHUNKED PROCESSING --- (long videos): split into overlapping chunks,
172     analyze them in parallel, offset timestamps back to full-video coordinates."""
173     ai_segments: List[Dict[str, Any]] = []
174     step = chunk_duration - chunk_overlap
175     chunk_starts = []
176     t = 0.0
177     while t < video_duration:
178         chunk_starts.append(t)
179         t += step
180     num_chunks = len(chunk_starts)
181     logger.info(
182         f"Video is {video_duration:.1f}s ? splitting into {num_chunks} overlapping
chunks "
183         f"({chunk_duration}s each, {chunk_overlap}s overlap).")
184
185     os.makedirs(chunks_dir, exist_ok=True)
186
187     def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list, dict]:
188         chunk_end = min(chunk_start + chunk_duration, video_duration)

```

15. user (2026-06-21T08:30:59.822Z)

```

300     lbl = f"Chunk {i+1}/{num_chunks}"
301     m = all_metrics.get(lbl, {})
302     sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
303     upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
304     prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
305     gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
306     seg = m.get('segments_found', 0) if m else 0
307     summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} | {gen:<8}

```

```

| {seg}")
308         summary_lines.append("=" * 60)
309         logger.info("\n".join(summary_lines))
310
311         # Clean up local temp files
312         cleanup_temp_files()
313         try:
314             os.rmdir(chunks_dir)
315         except Exception:
316             pass
317
318         logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
319
320         return ai_segments
321
322     def _process_single(
323         self,
324         video_id: str,

```

16. assistant (2026-06-21T08:31:03.968Z)

In `gemini.py`, ``chunks_dir`` is created at line 540 via ``_chunk_workdir(video_id, output_base(self.config))``, passed into ``_process_chunked``, `makedirs` at 186, `rmdir` at 314 ? same variable, consistent. The ``temp_trimmed_path`` create/remove both use ``base``. Good.

Now let me check the ``self.config`` attribute is consistently the same config object across all three segmenters, and confirm there's no separate typed-config path. Let me check how ``self.config`` is set and what ``self.get`` it supports.

17. user (2026-06-21T08:31:06.056Z)

```

backend/pipeline/segmenters/base.py:9:         self.config = config
backend/pipeline/segmenters/gemini.py:77:         sport_name = self.config.get("sport",
"badminton")
backend/pipeline/segmenters/gemini.py:85:         idx_cfg = self.config.get("indexing", {})
backend/pipeline/segmenters/gemini.py:86:         provider_name = self.config.get("ai_provider",
idx_cfg.get("ai_provider", "gemini"))
backend/pipeline/segmenters/gemini.py:115:         debug_duration = self.config.get("indexing",
{}).get("debug_duration")
backend/pipeline/segmenters/gemini.py:122:         base = output_base(self.config)
backend/pipeline/segmenters/gemini.py:249:         max_workers = max(1,
int(self.config.get("indexing", {}).get("ai_max_workers", 5)))
backend/pipeline/segmenters/gemini.py:370:         idx_cfg = self.config.get("indexing", {})
backend/pipeline/segmenters/gemini.py:449:         stitch_cfg = self.config.get("stitching", {})
backend/pipeline/segmenters/gemini.py:510:         model_name = self.config.get("ai_model",
self.config.get("indexing", {}).get("ai_model", "gemini-2.5-flash"))
backend/pipeline/segmenters/gemini.py:529:         idx_cfg = self.config.get("indexing", {})
backend/pipeline/segmenters/gemini.py:540:         chunks_dir = _chunk_workdir(video_id,
output_base(self.config))
backend/pipeline/segmenters/trajectory_hybrid.py:146:         sport_name =
self.config.get("sport", "badminton")
backend/pipeline/segmenters/trajectory_hybrid.py:153:         idx_cfg =
self.config.get("indexing", {})
backend/pipeline/segmenters/trajectory_hybrid.py:160:         provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend/pipeline/segmenters/trajectory_hybrid.py:167:         model_name =
self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))
backend/pipeline/segmenters/trajectory_hybrid.py:238:         out_dir =
os.path.join(output_base(self.config), self.family)
backend/pipeline/segmenters/trajectory_hybrid.py:314:         handoff_dir =
os.path.join(output_base(self.config), f"chunks_{provider_name}_handoff")
backend/pipeline/segmenters/yolo_hybrid.py:66:         sport_name = self.config.get("sport",
"badminton")
backend/pipeline/segmenters/yolo_hybrid.py:74:         idx_cfg = self.config.get("indexing", {})

```

```

backend/pipeline/segmenters/yolo_hybrid.py:75:         provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend/pipeline/segmenters/yolo_hybrid.py:76:         model_name = self.config.get("ai_model",
idx_cfg.get("ai_model", "gemini-2.5-flash"))
backend/pipeline/segmenters/yolo_hybrid.py:269:         gemini_dir =
os.path.join(output_base(self.config), f"chunks_{provider_name}_handoff")
backend/pipeline/segmenters/yolo_hybrid.py:375:         idx_cfg = self.config.get("indexing", {})
backend/pipeline/segmenters/yolo_hybrid.py:376:         sport_profile =
sport_registry.get(self.config.get("sport", "badminton"))
backend/pipeline/segmenters/yolo_hybrid.py:377:         provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend/pipeline/segmenters/yolo_hybrid.py:378:         model_name = self.config.get("ai_model",
idx_cfg.get("ai_model", "gemini-2.5-flash"))
backend/pipeline/segmenters/yolo_hybrid.py:408:         chunks_dir =
os.path.join(output_base(self.config), "chunks_yolo")

```

18. user (2026-06-21T08:31:06.439Z)

```

1      from abc import ABC, abstractmethod
2      from typing import List, Dict, Any, Type, Optional
3      from backend.infrastructure.database import Database
4      from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6      class BasePlaySegmenter(ABC):
7          def __init__(self, db: Database, config: Dict[str, Any]):
8              self.db = db
9              self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class

```

```

{segmenter_cls.__name__}: {e}") from e
46         self._segmenters[name] = segmenter_cls
47
48     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49         return self._segmenters.get(name)
50
51     def list_available(self) -> Dict[str, str]:
52         """Returns a dict of registered segmenter names and their descriptions."""
53         res = {}
54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

19. assistant (2026-06-21T08:31:09.246Z)

`self.config` is used throughout with `.get(...)` and is consistently the same config object the segmenters already read from. Now let me verify the typed AppConfig `.get` shim behavior and the `output.output\_dir` field ? this is critical for findings #1 and #4.

20. user (2026-06-21T08:31:10.846Z)

```

228:class OutputConfig(BaseModel):
233:    output_dir: str = "output"
303:    # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
321:    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
352:class AppConfig(BaseModel):
367:    output: OutputConfig = Field(default_factory=OutputConfig)
386:    def get(self, key: str, default: Any = None) -> Any:

```

21. user (2026-06-21T08:31:12.080Z)

```

backend\config\models.py:386:    def get(self, key: str, default: Any = None) -> Any:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:387-
_MISSING = object()
backend\config\models.py:388-        val: Any = _MISSING
backend\config\models.py:389-        if key in type(self).model_fields or hasattr(self, key):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:390-            val
= getattr(self, key, _MISSING)
backend\config\models.py:391-        if val is _MISSING:
backend\config\models.py:392-            # Legacy convenience: find a leaf key inside any nested
section. Sections are
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:393-            #
discovered dynamically from the model fields, so a future top-level section
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:394-            #
(e.g. `audio`, `jobs`) is covered automatically without editing a hardcoded list.
backend\config\models.py:395-            for fname in type(self).model_fields:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:396-            section = getattr(self, fname, None)
backend\config\models.py:397-            if isinstance(section, BaseModel) and
hasattr(section, key):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:398-            val = getattr(section, key)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:399-            break
backend\config\models.py:400-        if val is _MISSING:

```

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-401-
return default
backend\config\models.py-402-         if isinstance(val, BaseModel):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-403-
return val.model_dump(mode="json")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-404-         return
val
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-405-
backend\config\models.py-406-     def __getitem__(self, key: str) -> Any:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-407-         val =
self.get(key)
backend\config\models.py-408-         if val is None and key not in type(self).model_fields:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-409-             #
Re-raise like a real dict for unknown top-level
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-410-
raise KeyError(key)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-411-         return
val
--
backend\config\loader.py:210: def get_default_config() -> dict[str, Any]:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-211-     """Return
defaults as a plain dict (for transitional use only)."""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-212-     return
_get_builtin_defaults()

```

22. assistant (2026-06-21T08:31:16.723Z)

This is the key for finding #4. Let me trace exactly what `config.get("output", {})`.get("output_dir")` does on a typed `AppConfig`:

- `config.get("output", {})` ? `"output"` is in `model_fields` ? returns `getattr(self, "output")` which is an `OutputConfig` (a `BaseModel`) ? at line 402-403, since it's a `BaseModel`, returns `val.model_dump(mode="json")` ? a `**plain dict**`.
- Then `.get("output_dir")` on that dict ? returns the string value.

So for typed `AppConfig`, `output_base` returns the dict's `output_dir`. Let me confirm `OutputConfig` and check whether `model_dump` could omit `output_dir` or whether there are validation/alias concerns. Let me read the `OutputConfig` and `AppConfig` definitions.

23. user (2026-06-21T08:31:18.145Z)

```

228     class OutputConfig(BaseModel):
229         default_highlights_name: str = "highlights.mp4"
230         db_name: str = "sports_indexer.db"
231         # Directory where the DB, highlight reels, and processed clips are written.
232         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
233         output_dir: str = "output"
234
235
236     class WatermarkConfig(BaseModel):
237         """Branding overlay burned into each highlight clip during the per-clip re-encode
238         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
239         (under `stitching.watermark` in config.json) to turn it off. The text is fully
240         configurable. The concat stage stays stream-copy (-c copy)."""
241         enabled: bool = True
242         text: str = "Generated by Khelsutra.guru"
243         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
244         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
245         font: str = "Sans" # fontconfig family used when font_path is empty
246         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
247         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
248         box: bool = True # faint backing box behind the text for legibility
249         margin: int = Field(default=24, ge=0) # px inset from the chosen corner

```

```

250         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
251
252
253     class StitchingConfig(BaseModel):
254         begin_padding: float = 0.5
255         end_padding: float = 0.5
256         use_cache: bool = False
257         min_shot_count: int = 1

```

24. user (2026-06-21T08:31:18.708Z)

```

352     class AppConfig(BaseModel):
353         """Root configuration object.
354
355         All runtime configuration should be accessed via an instance of this class
356         (or the loader) rather than raw dict lookups.
357         """
358
359         sport: Sport = "badminton"
360         ai_provider: str = "gemini"
361         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
362         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
363         ai_model: str = "gemini-flash-latest"
364
365         validation: ValidationConfig = Field(default_factory=ValidationConfig)
366         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
367         output: OutputConfig = Field(default_factory=OutputConfig)
368         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
369         security: SecurityConfig = Field(default_factory=SecurityConfig)
370         logging: LoggingConfig = Field(default_factory=LoggingConfig)
371         storage: StorageConfig = Field(default_factory=StorageConfig)
372         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
373
374         # Allow extra keys during transition so existing config.json files do not break.
375         model_config = {
376             "extra": "allow",
377             "validate_assignment": True,
378         }
379
380         # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
381         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
382         # attribute access / the typed config. A nested *section* is returned as a plain
dict
383         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
384         # section model instead breaks it (a model has no .get), which bit every validator
on
385         # real runs. Scalar leaves are returned as-is.
386         def get(self, key: str, default: Any = None) -> Any:

```

25. assistant (2026-06-21T08:31:24.979Z)

Confirmed. The `.get` shim returns model_dump(mode="json")` of the section as a plain dict, so config.get("output", { }).get("output_dir")` returns "output"` (the default) on a fresh AppConfig. The output_dir` field is always present.`

One edge case for finding #4: what if `config` is a plain dict where config.get("output", { })` returns a non-dict (e.g., legacy config with output` as a string)? Then .get("output_dir")` would raise AttributeError` ? but that's caught by the try/except Exception` ? falls back to "output"`. Good, the bare-except` covers it.`

Now let me verify the critical question for finding #1: how does `--output-dir` flow into`

config, and could the config passed to segmenters EVER lack a coherent `output` section such that the default diverges? Let me check how segmenters are instantiated (what config object is passed) in the real pipeline.

26. user (2026-06-21T08:31:26.117Z)

```
backend/main.py:249:         # report.output_dir). Never raises. Surface the paths in the
response.
backend/main.py:339:     (which only writes into output_dir; the browser previously got back an
unreachable
backend/main.py:347:     output_dir = getattr(getattr(global_config, "output", None),
"output_dir", "output") or "output"
backend/main.py:348:     path = os.path.join(output_dir, name)
backend/main.py:350:     path = resolve_under_root(path, output_dir)
backend/main.py:418:     output_dir = getattr(getattr(global_config, "output", None),
"output_dir", "output") or "output"
backend/main.py:419:     compiler = VideoCompiler(output_dir,
begin_padding=stitching.begin_padding,
backend/main.py:505:     parser.add_argument("--output-dir", default="output", help="Directory
where processed clips/highlights are saved")
backend/main.py:546:         out_dir = args.output_dir
backend/main.py:578:     # on the typed model (OutputConfig.output_dir), so every consumer ?
including
backend/main.py:580:     global_config.output.output_dir = args.output_dir
backend/main.py:581:     os.makedirs(global_config.output.output_dir, exist_ok=True)
backend/main.py:583:     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
```

27. user (2026-06-21T08:31:27.601Z)

```
backend/main.py:32:from backend.config import load_config as load_app_config, AppConfig,
StitchingConfig # new typed config
backend/main.py:64:global_config: Optional[AppConfig] = None # now typed (Pydantic model)
backend/main.py:85:# New code should import load_app_config / AppConfig from backend.config
directly.
backend/main.py:169:     # typed AppConfig: attribute access for the default segmenter (with safe
fallback)
backend/main.py:212:         segmenter = segmenter_cls(db_instance, global_config)
backend/main.py:214:         result = segmenter.process_video(video_id, local_video,
req.player_pool, req.max_frames)
backend/main.py:251:         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
backend/main.py:261:def process_video(req: ProcessRequest):
backend/main.py:575:     global_config = load_app_config() # returns typed AppConfig
backend/main.py:646:         segmenter = segmenter_cls(db_instance, global_config)
backend/main.py:647:         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
backend/main.py:662:         db=db_instance, video_id=video_id,
video_path=args.video_path, config=global_config,
```

28. assistant (2026-06-21T08:31:34.545Z)

This is the crucial confirmation for finding #1. At line 580, `--output-dir` is written to `global\_config.output.output\_dir`, and at lines 212/646 the segmenter is constructed with `global\_config` (the typed AppConfig). So:

- **Default run** (no `--output-dir`, or `--output-dir output`):
`global\_config.output.output\_dir == "output" ?` `output\_base()` returns `"output" ?` byte-identical. ?
- Crucially: `output\_base()` reads `output.output\_dir`, which is **exactly the same field** as `main.py` already used for the DB path, highlight reels, etc. So M3's scratch dirs now consistently follow the same `--output-dir` as everything else. This is the latent-bug fix and it's coherent.

Now let me run the new test file and the broader segmenter test suite to verify green, and check whether yolo_hybrid / gemini call sites are tested. Let me also confirm there are no OTHER ``"output"`` literals left in these 3 files beyond what the diff touched (e.g., a ``wasb_hybrid`` subclass or shared helper). Let me check wasb_hybrid and the broader segmenters dir.

29. user (2026-06-21T08:31:37.058Z)

```
backend\pipeline\segmenters\gemini.py:17:def _chunk_workdir(video_id: str, base: str = "output")
-> str:
```

30. user (2026-06-21T08:31:44.326Z)

```
..... [100%]
7 passed in 5.05s
```

31. assistant (2026-06-21T08:31:48.675Z)

All 7 tests pass, and the only remaining ``"output"`` literal in the segmenters dir is the intentional ``base="output"`` default param of ``_chunk_workdir`` (which is byte-identical-by-design). Let me double-check there are no other hardcoded scratch literals using a different spelling (e.g., ``"output"`` built via a variable, or ``tempfile`` / direct string concatenation) in these 3 files, and confirm wasb_hybrid inherits trajectory_hybrid's now-fixed site.

32. user (2026-06-21T08:31:51.321Z)

```
=== wasb class hierarchy ===
12:lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
23:class WasbHybridSegmenter(TrajectoryHybridSegmenter):
```

33. user (2026-06-21T08:31:53.122Z)

No matches found

34. assistant (2026-06-21T08:31:58.565Z)

Confirmed: ``WasbHybridSegmenter`` subclasses ``TrajectoryHybridSegmenter`` and adds no scratch literals of its own, so it inherits the fixed ``output_base()`` site ? which is exactly what the test exercises. No other hardcoded scratch literals (no ``tempfile``, no ``"./"``, no other ``os.path.join("output"...)``) remain in the 3 files.

Now let me verify finding #5 thoroughly: the test only exercises the `**wasb (trajectory)**` call site at a real segmenter call. The gemini and yolo_hybrid call sites are NOT exercised at a real call site ? only ``_chunk_workdir`` (a gemini helper) is tested in isolation, and yolo_hybrid has zero coverage of its two new sites. Let me confirm what the existing test suite covers for yolo/gemini, and run the segmenter tests to ensure no regression.

35. user (2026-06-21T08:32:01.627Z)

```
tests/test_cleanup_caches.py
tests/test_config.py
tests/test_config_deployment.py
tests/test_database.py
tests/test_eval_batch.py
tests/test_eval_harness.py
tests/test_motion_segmenter.py
tests/test_yolo_hybrid.py
tests/__pycache__/test_cleanup_caches.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_config.cpython-313-pytest-9.0.3.pyc
=== gemini handoff/chunk tests ===
tests/test_gemini.py
tests/test_output_base.py
tests/__pycache__/test_gemini.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_gemini.cpython-313.pyc
tests/__pycache__/test_output_base.cpython-313-pytest-9.0.3.pyc
```


36. user (2026-06-21T08:32:20.878Z)

```
..... [ 36%]  
..... [ 72%]  
..... [100%]  
199 passed, 913 deselected in 17.00s
```

37. assistant (2026-06-21T08:32:25.148Z)

199 passed. Let me check what `test\_yolo\_hybrid.py` and `test\_gemini.py` already cover ? specifically whether they'd catch a regression in the new `output\_base()` sites (they pass `os.makedirs` patches but may assert nothing about the path). This determines whether the M3 yolo/gemini sites have *any* effective default-path coverage.

38. user (2026-06-21T08:32:28.066Z)

No matches found

39. user (2026-06-21T08:32:29.073Z)

```
tests/test_yolo_hybrid.py:66: ], {})  
tests/test_yolo_hybrid.py:72: config = {  
tests/test_yolo_hybrid.py:85: segmenter = HybridYoloSegmenter(db_mock, config)  
tests/test_yolo_hybrid.py:108: config = {"indexing": {"use_gpu": False}}  
tests/test_yolo_hybrid.py:110: segmenter = HybridYoloSegmenter(db_mock, config)  
tests/test_yolo_hybrid.py:134: ], {})  
tests/test_yolo_hybrid.py:140: config = {  
tests/test_yolo_hybrid.py:152: segmenter = HybridYoloSegmenter(db_mock, config)  
tests/test_yolo_hybrid.py:166: config = {"indexing": {"min_action_window_duration": 1.0}}  
tests/test_yolo_hybrid.py:167: segmenter = HybridYoloSegmenter(MagicMock(), config)  
tests/test_yolo_hybrid.py:202: ], {})  
tests/test_yolo_hybrid.py:209: config = {  
tests/test_yolo_hybrid.py:221: segmenter = HybridYoloSegmenter(db_mock, config)  
tests/test_yolo_hybrid.py:253: ], {})  
tests/test_yolo_hybrid.py:259: config = {  
tests/test_yolo_hybrid.py:272: segmenter = HybridYoloSegmenter(db_mock, config)  
tests/test_gemini.py:34: config = {  
tests/test_gemini.py:41: segmenter = GeminiPlaySegmenter(db_mock, config)  
tests/test_gemini.py:63: return [{"start_time": 10.0, "end_time": 20.0}], {}  
tests/test_gemini.py:65: return [{"start_time": 30.0, "end_time": 40.0}], {}  
tests/test_gemini.py:73: config = {  
tests/test_gemini.py:80: segmenter = GeminiPlaySegmenter(db_mock, config)  
tests/test_gemini.py:106: config = {"indexing": {"chunk_duration": 150, "chunk_overlap": 10,  
*(indexing_overrides or {}))}  
tests/test_gemini.py:107: segmenter = GeminiPlaySegmenter(db_mock, config)  
tests/test_gemini.py:182: config = {"indexing": {"chunk_duration": 300, "chunk_overlap": 30}}  
tests/test_gemini.py:183: segmenter = GeminiPlaySegmenter(MagicMock(), config)
```

40. assistant (2026-06-21T08:32:35.537Z)

The existing yolo/gemini tests pass dict configs *without* an `output` section and assert nothing about the scratch path ? they patch `os.makedirs`. So they exercise the new `output\_base(self.config)` code path (proving it doesn't crash on a config lacking `output` ? returns `output`), but they do *not* assert the resulting path equals `output/chunks\_yolo` etc. So those sites have *execution* coverage (no-crash) but not *path-assertion* coverage. Only the wasb/trajectory site is pinned to an exact path. This confirms finding #5.

I have everything needed. Let me do one final confirmation on the create/cleanup pairing in yolo_hybrid (the rmdir at 314/424) and trajectory_hybrid (rmdir 350), reading the exact lines to be 100% sure the rmdir uses the same variable created earlier.

41. user (2026-06-21T08:32:37.427Z)

266

""Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask

```

267         the provider for semantic boundaries, in parallel. Verbatim move."""
268         gemini_segments = []
269         gemini_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")
270         os.makedirs(gemini_dir, exist_ok=True)
271
272         logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
273
274         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
275             w_start = max(0, window["start"] - handoff_padding)
276             w_end = min(video_duration, window["end"] + handoff_padding)
277             w_dur = w_end - w_start
278
279             w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
280             # Re-encode short handoff clips to avoid keyframe-boundary drift
281             success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
282             if not success:
283                 return []
284
285             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
286             segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
287
288             # Coordinate mapping ? shift local timestamps back to full-video time
289             valid_segments = []
290             for segment in segments:
291                 try:
292                     segment["start_time"] = float(segment["start_time"]) + w_start
293                     segment["end_time"] = float(segment["end_time"]) + w_start
294                     # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
295                     segment["_candidate_id"] = candidate_ids[wi]
296                     valid_segments.append(segment)
297                 except (ValueError, TypeError, KeyError) as e:
298                     logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")
299
300                 try:
301                     os.remove(w_path)
302                 except OSError:
303                     pass
304
305             return valid_segments
306
307         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
308             futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
309             # Iterate in submission order to preserve temporal ordering
310             for future in futures:
311                 gemini_segments.extend(future.result())
312
313             try:
314                 os.rmdir(gemini_dir)
315             except OSError:
316                 pass
317

```

42. user (2026-06-21T08:32:38.006Z)

```

308         }
309         for wi in handoff_indices
310     ]

```

```

311         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
312                     "(no AI handoff).", len(ai_segments))
313     else:
314         handoff_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")
315         os.makedirs(handoff_dir, exist_ok=True)
316         logger.info("Phase 3: %s validation on %d candidate windows...",
317                     provider_name, len(handoff_indices))
318
319         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
320             w_start = max(0, window["start"] - handoff_padding)
321             w_end = min(video_duration, window["end"] + handoff_padding)
322             w_dur = w_end - w_start
323             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
324             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
325                 return []
326             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
327             segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
328                                                  label=f"Handoff {wi+1}")
329             valid = []
330             for seg in segments:
331                 try:
332                     seg["start_time"] = float(seg["start_time"]) + w_start
333                     seg["end_time"] = float(seg["end_time"]) + w_start
334                     seg["_candidate_id"] = candidate_ids[wi]
335                     valid.append(seg)
336                 except (ValueError, TypeError, KeyError) as e:
337                     logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
338             try:
339                 os.remove(w_path)
340             except OSError:
341                 pass
342             return valid
343
344         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
345             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
346                        for wi in handoff_indices]
347             for future in futures:
348                 ai_segments.extend(future.result())
349             try:
350                 os.rmdir(handoff_dir)
351             except OSError:
352                 pass

```

43. assistant (2026-06-21T08:33:09.144Z)

Create/cleanup pairing fully confirmed in all sites:

- ****yolo_hybrid****: `gemini\_dir` (269 makedirs 270 ? rmdir 314); `chunks\_dir` (408 makedirs 409 ? rmdir 424) ? same local var.
- ****trajectory_hybrid****: `handoff\_dir` (314 makedirs 315 ? rmdir 350) ? same local var.

`out\_dir` (238) is passed to `runner.run\_predict` which owns its own makedirs, no rmdir here (no orphan risk introduced by M3).

- ****gemini****: `chunks\_dir` (540 ? makedirs 186 ? rmdir 314); `base`/`temp\_trimmed\_path` (124 makedirs ? 152/127 remove) ? same vars.

`output\_base()` is a pure function of the same `self.config` object, so creation and cleanup compute the ****identical**** base every time within a run (no cleanup pointing at a literal `"output"`). No orphan/rmdir-mismatch risk.

I have all findings grounded. Here is the review.

M3 Review ? COMPUTE_DECOUPLED_SERVING: configurable output/scratch base

****Verdict: CLEAN BILL ? ship it.**** No blockers, no majors. M3 does exactly what it claims: routes the 6 hardcoded scratch/staging/temp sites through ``output_base(config)``, defaults to ``"output"`` byte-identically, and is effectively default-OFF (only a non-default ``--output-dir`` moves anything). All 7 new tests + 199 segmenter/config/path tests pass. Two minor/nit observations on test coverage and a doc nuance, no fix required to merge.

1. Byte-identical default ? PASS

- ``output_base()`` reads ``output.output_dir`` (``backend/utils/paths.py:16``), which defaults to ``"output"`` (``OutputConfig.output_dir``, ``backend/config/models.py:233``).
- In the real pipeline the segmenter receives the typed ``global_config`` (``backend/main.py:212,646``), and ``--output-dir`` is written to the **same** field ``output.output_dir`` (``backend/main.py:580``) that already governs the DB path, reels, and clips. So M3's scratch now follows the identical knob as everything else ? that's the latent bug it fixes, and the default path is unchanged.
- No code path returns a non-``"output"`` base by default: missing ``output`` section, missing key, empty string, ``None`` config, and a non-dict ``output`` value all fall back to ``"output"`` via ``out`` or ``"output" + the bare `except`` (``paths.py:15-19``). Verified by ``test_output_base_falls_back_to_output_on_missing_or_empty``.

2. Create/cleanup consistency ? PASS

Every ``makedirs/rmdir`` pair uses the **same local variable**, computed once from ``output_base(self.config)``, so cleanup can never point at a literal ``"output"`` while creation used a custom base:

- ``yolo_hybrid.py``: ``gemini_dir`` (270?314), ``chunks_dir`` (409?424).
- ``trajectory_hybrid.py``: ``handoff_dir`` (315?350). ``out_dir`` (238) has no ``rmdir`` here ? the ``DetectorRunner`` owns it; M3 introduces no orphan.
- ``gemini.py``: ``chunks_dir`` threaded 540?186(``makedirs``)?314(``rmdir``); ``base`/`temp_trimmed_path`` (124?127/152). All consistent.

``output_base`` is a pure function of ``self.config``, so create and cleanup always derive the identical base within a run. No orphan / failing-``rmdir`` risk.

3. Missed sites ? PASS

The only remaining ``"output"`` literal anywhere in ``backend/pipeline/segmenters/`` is the intentional ``base="output"`` default param of ``_chunk_workdir`` (``gemini.py:17``), which is byte-identical-by-design. No ``tempfile`/`mkdtemp`/`"/`/other`` ``os.path.join("output", ?)`` literals remain in the 3 files. ``WasbHybridSegmenter`` subclasses ``TrajectoryHybridSegmenter`` with no scratch literals of its own, so it inherits the fixed site. (``rally_seg_eval.py`` ``DEFAULT_MANIFEST`` correctly ignored per scope.)

4. Config access ? PASS

``config.get("output", {}).get("output_dir")`` works for both shapes:

- ****Typed AppConfig****: ``_.get("output", {})`` hits ``model_fields``, and since the section is a ``BaseModel`` it is returned as a plain dict via ``model_dump(mode="json")`` (``models.py:402-403``) ? so the chained ``_.get("output_dir")`` works. Confirmed by ``test_output_base_default_is_output(AppConfig())`` and ``...honors_custom_output_dir``.
- ****Legacy dict****: ordinary nested ``_.get``.
- No uncaught raise: any ``AttributeError`` (e.g. ``output`` is a string in a legacy dict) is swallowed by the bare ``except Exception`` ? ``"output"``.

5. Test gaps ? minor

- ****Minor ? only the wasb/trajectory call site is path-pinned.**** ``tests/test_output_base.py`` pins both default (``output/wasb``) and custom (``<custom>/wasb``) at a *real* segmenter call (``_run_wasb_capture_out_dir``, lines 81-90), which is solid. But the ``yolo_hybrid`` sites (``chunks_yolo``, ``chunks_{provider}_handoff``) and the ``gemini`` ``temp_trimmed`/`handoff`` sites get ****no path assertion**** ? only ``_chunk_workdir`` is unit-tested in isolation. The pre-existing ``test_yolo_hybrid.py`/`test_gemini.py`` pass dict configs with no ``output`` section and assert nothing about the path (they patch ``os.makedirs``), so those new sites have no-crash execution

coverage but not custom-base coverage. Risk is low (all sites use the identical one-liner), but a future regression on the yolo/gemini sites would not be caught. Fix (optional): add one yolo-style capture test mirroring `\_run\_wasb\_capture\_out\_dir` asserting `chunks\_yolo` follows a custom `output\_dir`.

- **Nit** ? the trajectory `out\_dir` is tested only via `run\_predict`'s 2nd positional arg. Fine and intentional, just noting the test depends on that arg position (`call\_args[0][1]`); a signature change to `run\_predict` would silently mis-target. Acceptable.

Net: correct, well-scoped, byte-identical-by-default, create/cleanup-coherent. The only gap is asymmetric test depth (wasb pinned, yolo/gemini not) ? a minor coverage nit, not a correctness defect.